

## TEST AUTOMATION PROJECT REPORT

# Ixigo Flight Booking Automation

Selenium WebDriver, pytest, Page Object Model, pytest-html and Allure

**Application under test:** ixigo flight booking web journey

**Automation stack:** Python, Selenium WebDriver, pytest, Page Object Model

**Execution browser:** Microsoft Edge, non-headless mode

**Report date:** 22 May 2026

**Result summary:** 24 Selenium checks documented from the combined flow and the full end-to-end booking flow

---

## 1. Synopsis

The project automates the main ixigo flight-booking path using Selenium WebDriver. The suite starts from the flight-search page, validates core search behaviour, applies practical result-page actions such as filtering and sorting, and then continues through booking, traveller details, add-ons, seat selection and the payment page. The goal is not only to click through screens, but to prove that the most important user journey still behaves correctly when handled by an automated test framework.

The work is organised as a Selenium pytest framework with page objects, reusable utilities, CSV-based test data, screenshots and HTML/Allure reporting. This keeps the test code readable and makes the result easier to review during a capstone-style evaluation.

## 2. Project Description

The application under test is the ixigo flight booking website. A normal user journey contains several moving parts: source and destination selection, calendar interaction, result loading, pop-up handling, price sorting, booking navigation, traveller form entry, add-ons and payment routing. These are exactly the areas where manual testing can become repetitive and where automated regression coverage is valuable.

The Selenium project was therefore built around two practical flows. The combined flow checks search, results, negative input handling and a booking smoke path. The end-to-end flow keeps the same browser session alive and follows the richer journey up to the payment page.

## 3. Objectives and Scope

- Automate the main user journey from launching ixigo to reaching the payment page.

- Validate search-page elements, source and destination selection, and flight-result availability.
- Check important result-page actions such as applying the non-stop filter and sorting by price.
- Cover negative scenarios where the user should not be allowed to continue with invalid search input.
- Use external CSV files for login, route and traveller data so that test data remains separate from test logic.
- Generate evidence through screenshots, logs, pytest HTML report output and Allure result files.

## 4. Test Environment

| Area            | Details                                                      |
|-----------------|--------------------------------------------------------------|
| Language        | Python                                                       |
| Automation tool | Selenium WebDriver                                           |
| Test runner     | pytest                                                       |
| Reporting       | pytest-html, Allure result attachments and saved screenshots |
| Browser         | Microsoft Edge, visible execution                            |
| Design pattern  | Page Object Model                                            |
| Data source     | CSV files for route, login and traveller data                |
| Main command    | pytest, using the options configured in pytest.ini           |

## 5. Framework Architecture

The framework is intentionally simple. The tests describe the scenario, the page classes perform browser actions, and the utility layer takes care of configuration, data reading, logging and screenshots. This separation matters because ixigo pages change state often; keeping selectors and actions in page objects makes maintenance easier than writing everything directly inside test methods.

| Layer     | Files / folders | Responsibility                                                                              |
|-----------|-----------------|---------------------------------------------------------------------------------------------|
| Tests     | tests/          | Contains ordered pytest scenarios for combined checks and the full booking journey.         |
| Pages     | pages/          | Encapsulates browser interactions for search, results, booking, add-ons, payment and login. |
| Utilities | utils/          | Reads config and CSV data, writes logs, and captures screenshots.                           |
| Data      | data/           | Stores flight routes, login mobile number and traveller details.                            |
| Reports   | reports/        | Stores pytest HTML output, screenshots and Allure result files.                             |

## 6. Test Data Strategy

The suite avoids hard-coding the main input values directly inside the test methods. Routes are read from flight\_routes.csv, traveller details come from traveller\_data.csv, and the login mobile number is read from login\_data.csv. This is a small detail, but it makes the framework feel closer to a real testing project because data can be changed without touching the automation logic.

|                           |                                                                               |
|---------------------------|-------------------------------------------------------------------------------|
| <b>flight_routes.csv</b>  | Source city, source code, destination city, destination code and travel date. |
| <b>traveller_data.csv</b> | Title, name, nationality, contact number, email, pincode and address.         |
| <b>login_data.csv</b>     | Mobile number used to begin the login flow before OTP entry.                  |

## 7. Execution and Reporting

The pytest configuration already defines the main reporting options. A normal test run writes a self-contained HTML report to reports/report.html, stores screenshots in reports/screenshots, and produces Allure result files in reports/allure-results. The HTML report is useful for quick submission review, while Allure gives a deeper drill-down view with attachments.

| Output         | Location                | Purpose                                                              |
|----------------|-------------------------|----------------------------------------------------------------------|
| HTML report    | reports/report.html     | Readable browser-based execution summary.                            |
| Screenshots    | reports/screenshots/    | Visual evidence captured during important steps and test completion. |
| Allure results | reports/allure-results/ | Raw result data and attachments for Allure report generation.        |
| Logs           | logs/automation.log     | Execution trace for debugging and review.                            |

## 8. Selenium Test Case Summary

**Result:** The Selenium evidence available in the project shows the combined flow and the richer full end-to-end flow. Together they cover 24 checks: 9 combined-flow scenarios and 15 end-to-end booking steps.

| No. | Suite    | Test case                 | Expected result                            | Status |
|-----|----------|---------------------------|--------------------------------------------|--------|
| 1   | Combined | Open ixigo page           | Application opens and title contains ixigo | Passed |
| 2   | Combined | Page elements are present | From, To and Search controls are visible   | Passed |

|    |          |                               |                                                           |        |
|----|----------|-------------------------------|-----------------------------------------------------------|--------|
| 3  | Combined | Search BOM to BLR flights     | Flight results load and count is greater than zero        | Passed |
| 4  | Combined | Flight count and book buttons | Flights and visible Book buttons are available            | Passed |
| 5  | Combined | Apply filter and sort         | Non-stop filter and price sorting keep usable results     | Passed |
| 6  | Combined | Print flight results          | Top results are printed to the logger                     | Passed |
| 7  | Combined | Same source and destination   | Invalid same-route search does not navigate to results    | Passed |
| 8  | Combined | Search without destination    | Search is blocked when destination is missing             | Passed |
| 9  | Combined | Booking flow smoke check      | Search to booking page is completed                       | Passed |
| 10 | E2E      | Open site                     | Flight search page opens in the same browser session      | Passed |
| 11 | E2E      | Login                         | Mobile number is submitted and OTP wait is handled        | Passed |
| 12 | E2E      | Search flights                | CSV route is searched and results are loaded              | Passed |
| 13 | E2E      | Apply non-stop filter         | Filtered results remain available                         | Passed |
| 14 | E2E      | Sort by price                 | Sorted results remain available                           | Passed |
| 15 | E2E      | Print results                 | At least one flight result is available to print          | Passed |
| 16 | E2E      | Click Book                    | Book action is triggered for the selected flight          | Passed |
| 17 | E2E      | Booking page                  | Booking URL/page state is detected                        | Passed |
| 18 | E2E      | Decline cancellation          | Free-cancellation prompt is handled                       | Passed |
| 19 | E2E      | Fill traveller                | Traveller form is filled from CSV data                    | Passed |
| 20 | E2E      | Continue and confirm          | Traveller details are confirmed                           | Passed |
| 21 | E2E      | Add-ons page                  | Add-ons page loads after booking confirmation             | Passed |
| 22 | E2E      | Select seat                   | Seat is selected when available, otherwise flow continues | Passed |
| 23 | E2E      | Skip to payment               | Add-ons are skipped and payment navigation begins         | Passed |
| 24 | E2E      | Payment page                  | Payment page is reached successfully                      | Passed |

## 9. Screenshot Evidence

The following screenshots are taken from the Selenium execution and are placed in the same order as the user journey. They show the automation moving through the major checkpoints rather than only listing the final pass status.

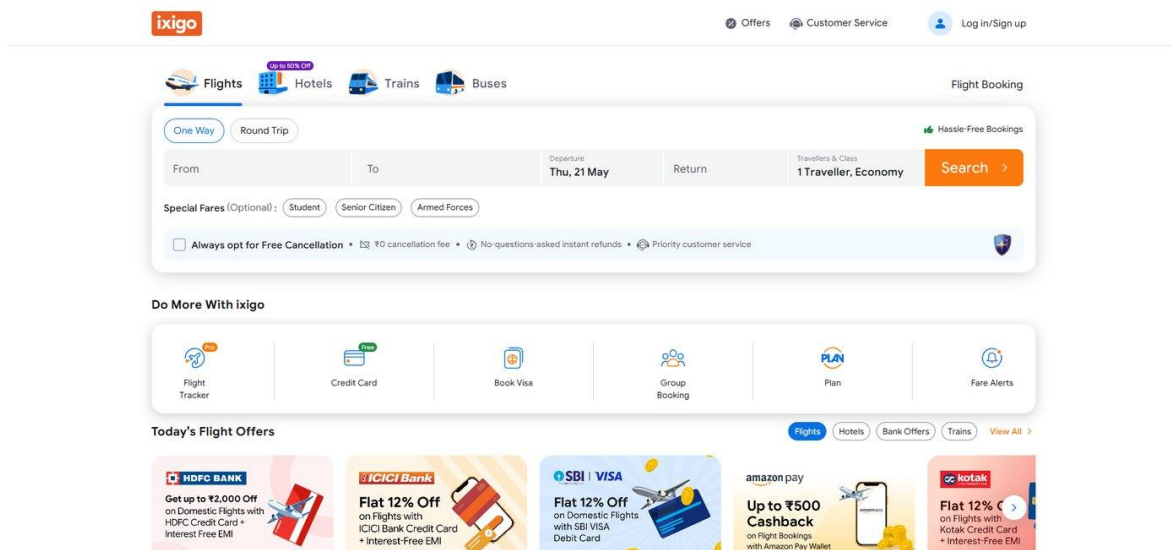


Figure 1. ixigo flight search page opened successfully.

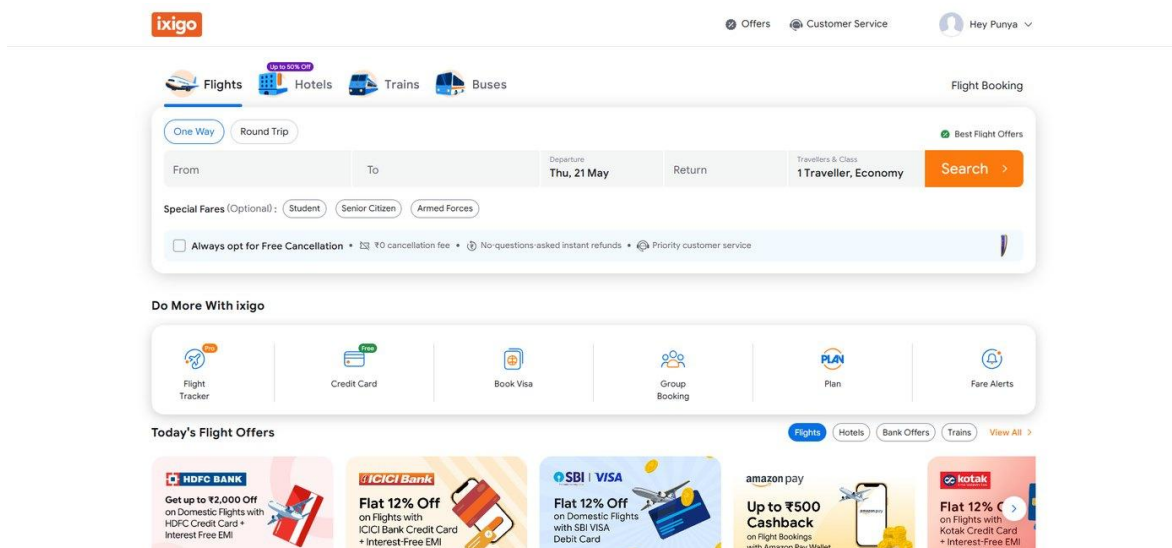


Figure 2. Login flow started with the mobile number from CSV test data.

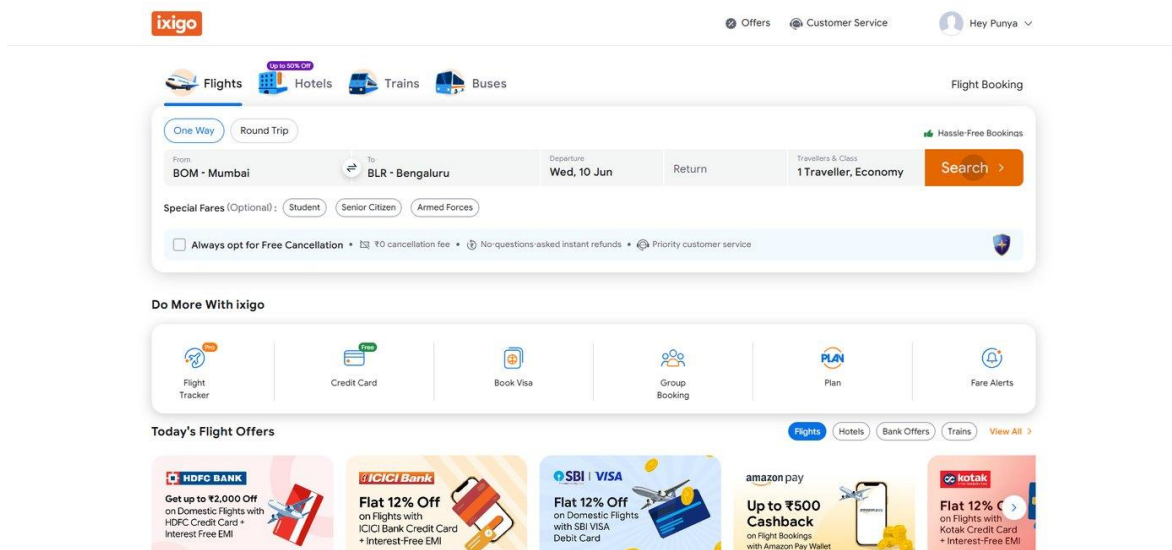


Figure 3. Search form completed before navigation to flight results.

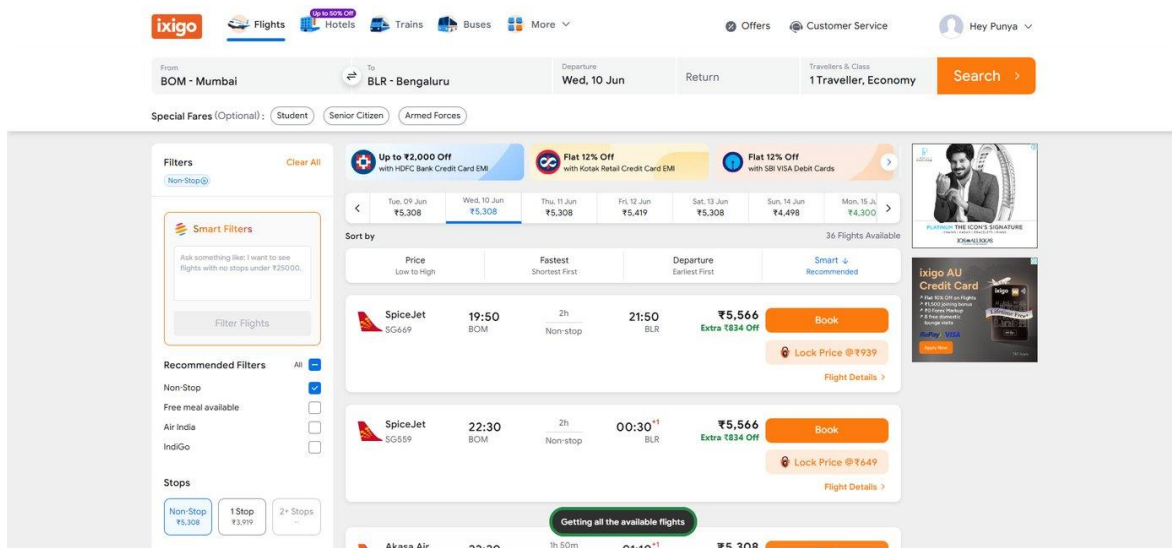


Figure 4. Non-stop filter applied on the results page.

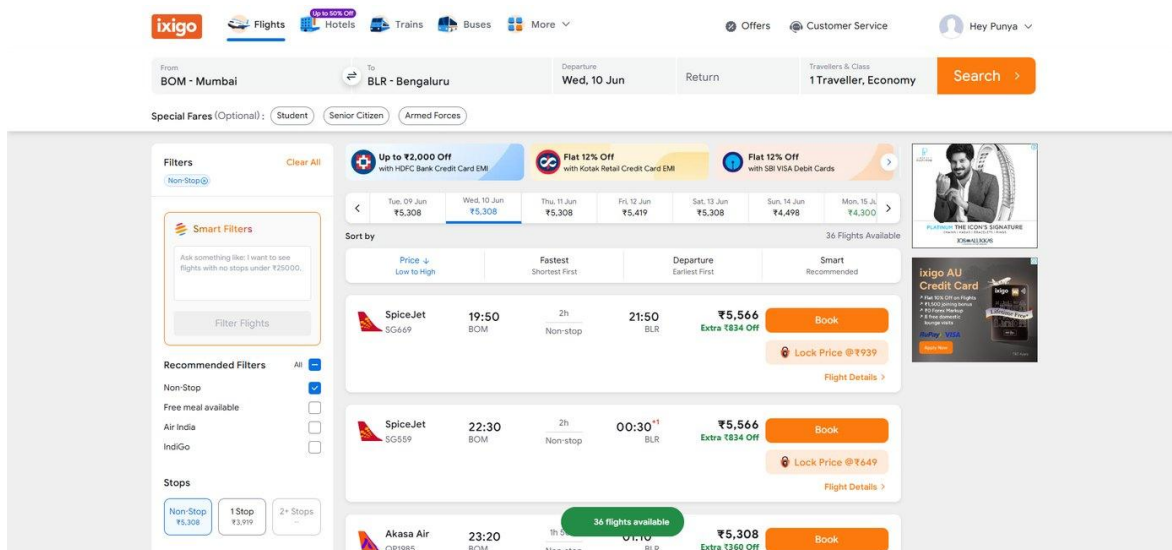


Figure 5. Results sorted by price after filtering.

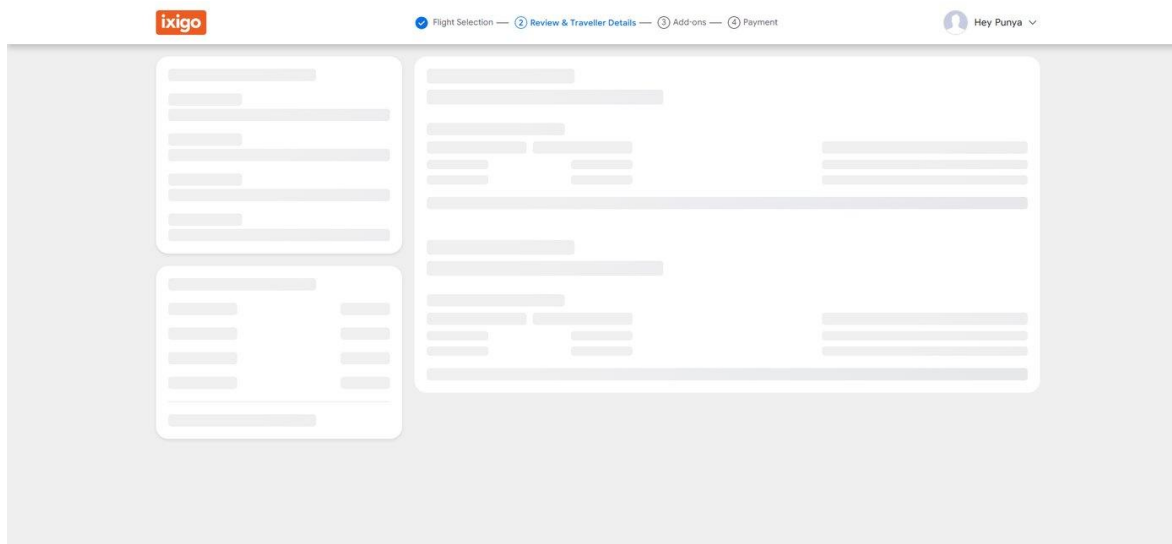


Figure 6. Cheapest available flight selected for booking.

**ixigo** Flight Selection — 2 Review & Traveller Details — 3 Add-ons — 4 Payment Hey Punya

**Offers For You**

Have a promo code? Redeem here.

- SALE** ₹834 Off  
Coupon applied! Enjoy an instant discount of ₹834. [Know More](#)
- IXRBLEMI** ₹667 Off  
Flat 12% Off up to ₹1,500 with RBL Bank Credit Card + Interest Free EMI. [Know More](#)
- SBVISADCD** ₹667 Off  
Flat 12% Off up to ₹1,500 with SBI VISA Debit Card. [Know More](#)

[View All Offers >](#)

**Fare Summary** 1 Traveller

|                     |                      |
|---------------------|----------------------|
| Fare Type           | Partially Refundable |
| Base Fare           | ₹4,200               |
| Taxes & Fees        | ₹1,366               |
| Instant Off         | -₹834                |
| <b>Total Amount</b> | <b>₹4,732</b>        |

**Mumbai → Bengaluru**  
Wed, 10 Jun • Non-stop • 2h • Economy

**SpiceJet 15G 669**

Wed, 10 Jun  
**19:50**  
BOM - Mumbai  
Mumbai Chhatrapati Shivaji International Airport  
Terminal 1

2h

Wed, 10 Jun  
**21:50**  
BLR - Bengaluru  
Kempegowda International Airport  
Terminal 1

Baggage  
Cabin : 7 kg per adult  
Check-in : 15 kg per adult

**Refund on Cancellation** [Cancellation & Rescheduling Policy >](#)

BOM - BLR

₹533 refund

Now 19:50 06 Jun 19:50 Departure 10 Jun 17:50

**Traveller Details**  
Choose from the saved list or add a new passenger

[Add](#) Please ensure that your name matches your govt. ID such as Aadhaar, Passport or Driver's License

₹4,732 ~~₹5,566~~ [Lock Price @₹1939](#) [Continue >](#)

Figure 7. Free-cancellation prompt handled during booking.

**ixigo** Flight Selection — 2 Review & Traveller Details — 3 Add-ons — 4 Payment Hey Punya

**Fare Summary** 1 Traveller

|                     |                      |
|---------------------|----------------------|
| Fare Type           | Partially Refundable |
| Base Fare           | ₹4,200               |
| Taxes & Fees        | ₹1,366               |
| Instant Off         | -₹834                |
| <b>Total Amount</b> | <b>₹4,732</b>        |

**GST Details**  
To claim credit of GST charged by airlines, please enter your GST details.

☐ I would like to add my GST Number.

**Billing Address**  
As per the latest govt. regulations, it's mandatory to provide your address.

Pincode: 560001 Address: 42 MG Road Bengaluru  
City: Bengaluru State: Karnataka

By clicking on continue, I confirm that I have read, understood, and agree with the [Fare Rules](#), [Privacy Policy](#) and [Terms of Use](#).

100% Safe Payment Process

[Verified by Visa](#) [SafeKey](#) [MasterCard SecureCode](#)

₹4,732 ~~₹5,566~~ [Lock Price @₹1939](#) [Continue >](#)

Figure 8. Traveller details filled from the CSV data file.



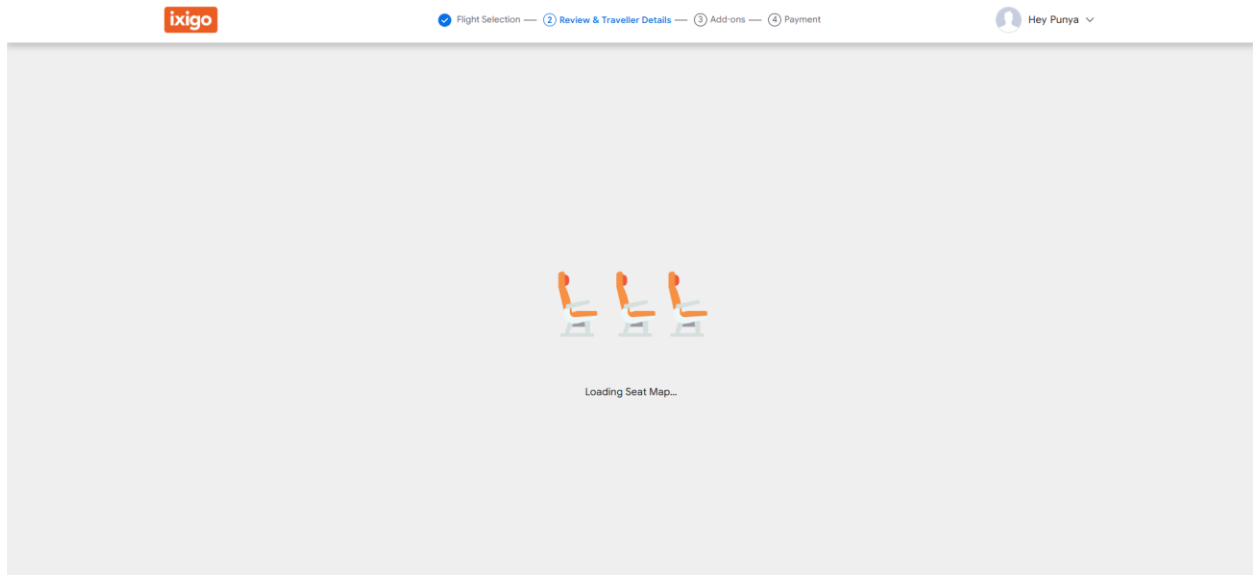


Figure 9. Add-ons page reached after traveller confirmation

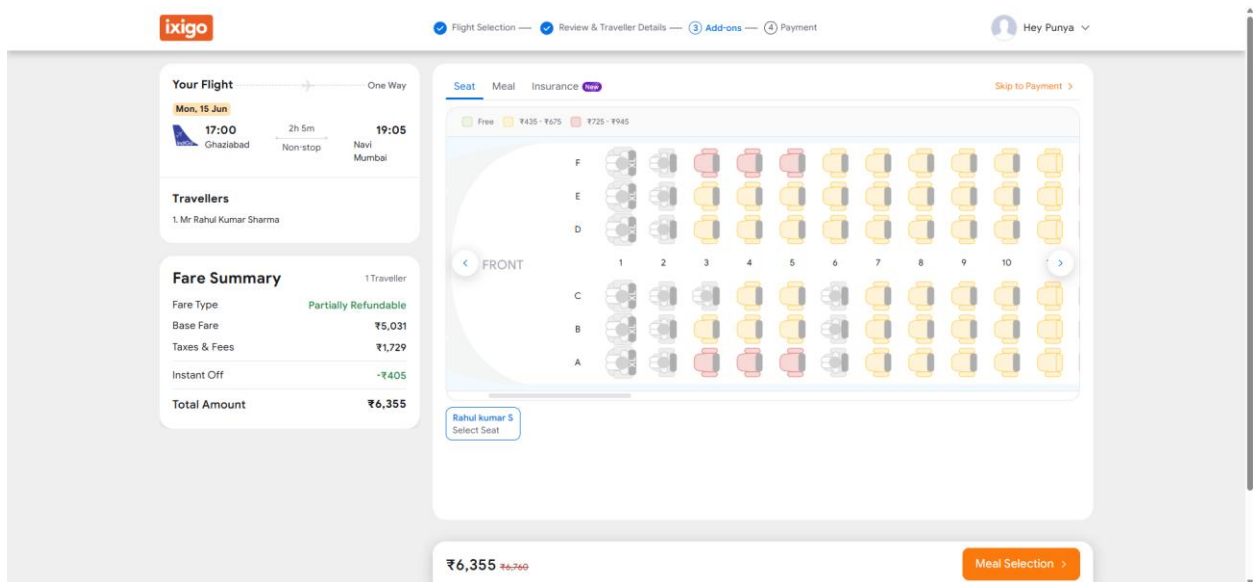


Figure 10. Seat selection step completed or skipped depending on availability.

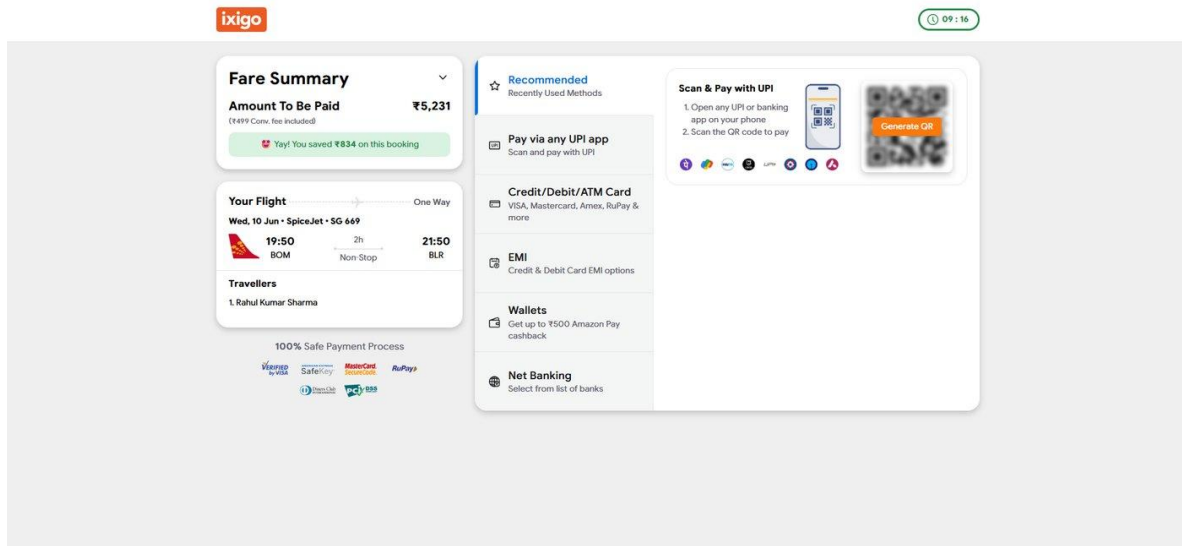


Figure 11. Payment page reached, completing the end-to-end flow.

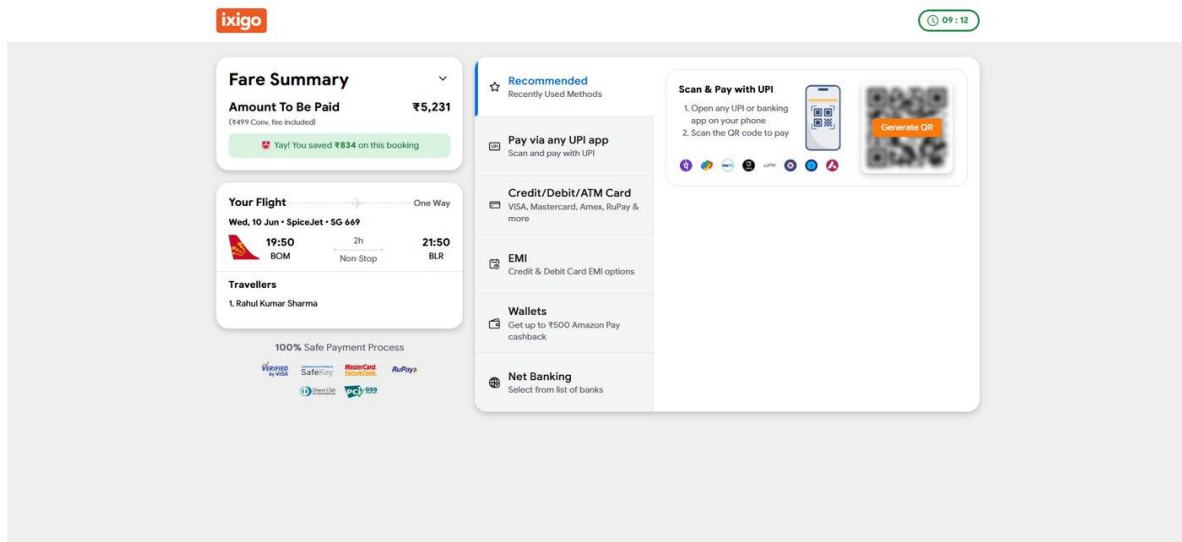


Figure 12. Final captured state after the full booking journey.

## 10. Observations

- The Page Object Model made the flow easier to read because test methods stayed close to business behaviour.
- The richer end-to-end file is useful for demonstration because the browser session remains continuous from login to payment.
- Negative tests add balance to the suite by proving that invalid input does not quietly move to the results page.
- Screenshots are captured at meaningful points, which makes the report easier to verify without replaying the full run.

## 11. Conclusion

The Selenium automation project successfully covers the important ixigo flight booking path and presents the result in a way that can be reviewed by both a technical evaluator and a project guide. The framework uses a sensible structure, external test data, browser-level validation, screenshots and report output. Overall, the project demonstrates practical web automation rather than a collection of isolated click scripts.

## TEST AUTOMATION PROJECT REPORT

# Ixigo Flight Booking Automation - BDD

Behavior Driven Development with Behave, Gherkin, Selenium WebDriver, POM, Allure and custom HTML reporting

**Application under test:** ixigo flight booking web journey

**Test design approach:** Behavior Driven Development using Gherkin scenarios

**BDD framework:** Behave feature files with Selenium-backed step definitions

**Automation pattern:** Page Object Model for browser/page actions

**Execution browser:** Microsoft Edge or Chrome, selected by environment hook

**Report date:** 23 May 2026

**Behaviour coverage:** 24 BDD scenarios across combined flow and full end-to-end booking flow

---

## 1. Synopsis

This project converts the ixigo Selenium automation into a BDD-style Behave framework. The tested behaviour remains the same: search flights, validate results, apply filter and sort actions, handle invalid search input, move into booking, enter traveller details, pass through add-ons and reach the payment page. The difference is in how the test intent is expressed. Instead of only Python test methods, the BDD version presents the flow through readable Gherkin scenarios and keeps the Selenium work inside step definitions.

The structure is deliberately close to the Selenium project so the evaluator can see that the framework changed while the business journey stayed consistent. The BDD code has two feature files and two step-definition folders: one for the combined flow and one for the richer full end-to-end booking flow.

## 2. Project Scope

- Represent the major ixigo user behaviours in Gherkin using Given, When and Then steps.
- Keep assertions inside step definitions so each Then step validates a real browser state.
- Reuse the Page Object Model for browser interaction rather than placing Selenium locators in feature files.
- Run the combined search, negative and booking smoke checks separately from the full E2E booking flow.
- Capture screenshots during important actions and after each scenario for evidence.

- Generate Allure result files and a custom local HTML report after the Behave run.

### 3. BDD Architecture

The code uses BDD and POM together. BDD describes the expected behaviour in feature files; POM organises the Selenium implementation behind those steps. This means the evaluator can read the scenario intent without seeing locators, while the automation code remains maintainable.

| Layer          | Actual code location                                                 | Responsibility                                                                              |
|----------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Feature files  | features/combined_flow.feature,<br>features/e2e_full_booking.feature | Define behaviours in readable Gherkin language.                                             |
| Combined steps | features/steps/combined_flow_steps/combined_steps.py                 | Search, result validation, filter, sort, negative scenarios and booking smoke checks.       |
| E2E steps      | features/steps/e2e_full_booking_steps/e2e_steps.py                   | Login, traveller details, add-ons, seat selection and payment checks.                       |
| Hooks          | features/environment.py                                              | Browser lifecycle, report cleanup, screenshots, HTML report generation and Allure opening.  |
| Page objects   | pages/                                                               | Encapsulate Selenium interactions for search, results, booking, add-ons, payment and login. |
| Data/utilities | data/, utils/                                                        | CSV test data, route reading, config reading, logging and screenshot capture.               |

### 4. Behaviour Files

The BDD project has two feature files. The combined feature checks the smaller behaviours that would normally be reviewed as separate regression checks. The full end-to-end feature keeps the booking journey in order so it looks like the Selenium E2E flow, but described as business behaviour.

| Feature                       | Scenario count | Main behaviours                                                                                                                                                                                |
|-------------------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ixigo combined flow           | 9              | Open page, verify controls, search BOM to BLR, verify book buttons, filter/sort, print results, invalid source/destination, missing destination and booking-page smoke flow.                   |
| Ixigo full end-to-end booking | 15             | Open site, login, search random CSV route, filter, sort, print results, book, booking page, cancellation prompt, traveller details, add-ons, seat selection, skip to payment and payment page. |

### 5. Assertion Strategy

The feature files are intentionally readable and do not contain Python assertions. The assertions are implemented in the step-definition files, which is the normal Behave pattern. The Gherkin says what should happen; the Python step code proves it using Selenium and page objects.

| BDD check                  | Where it is asserted | Validation performed                                                |
|----------------------------|----------------------|---------------------------------------------------------------------|
| Page loaded                | combined_steps.py    | Browser title contains ixigo.                                       |
| Search controls visible    | combined_steps.py    | From, To and Search elements are present.                           |
| Matching results displayed | combined_steps.py    | Results load and flight count is greater than zero.                 |
| Visible flights remain     | combined_steps.py    | Flight cards and Book buttons are visible.                          |
| Prices sorted              | combined_steps.py    | First visible prices are in ascending order when prices are parsed. |
| Booking page displayed     | combined_steps.py    | Booking URL or traveller/contact content is detected.               |
| Traveller form filled      | e2e_steps.py         | CSV traveller details are entered and form fill returns true.       |
| Payment page displayed     | e2e_steps.py         | Payment page wait condition returns true.                           |
| Negative route checks      | combined_steps.py    | Invalid route does not navigate to accepted result URL.             |

## 6. Execution Design

The hook file starts one browser per feature, matching the Selenium-style ordered flow. Scenarios tagged as isolated can still use their own browser, which keeps negative checks from damaging the main journey. Before every run, report folders and logs are cleaned so the output represents the latest execution.

- Nested step modules are enabled in behave.ini, so Behave loads both step-definition folders directly.
- The default formatter writes Allure result files to reports/allure-results.
- environment.py patches Allure behaviour labels, builds a custom HTML report and opens it after execution.
- If the Allure CLI is unavailable, the local HTML report still opens and the test run does not fail because of reporting.
- Screenshots are captured both during key actions and at scenario completion.

## 7. Test Environment

| Area                      | Details                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------|
| Language                  | Python                                                                                     |
| Test design approach      | Behavior Driven Development with Given, When and Then scenarios                            |
| BDD framework / runner    | Behave                                                                                     |
| Browser automation tool   | Selenium WebDriver                                                                         |
| Automation design pattern | Page Object Model                                                                          |
| Step discovery            | Nested step modules enabled in behave.ini                                                  |
| Reports                   | Allure result files, generated Allure report when CLI is installed, and custom report.html |

|         |                                                                |
|---------|----------------------------------------------------------------|
| Browser | Configured browser with Edge/Chrome fallback in environment.py |
| Data    | CSV files for route, login and traveller input                 |

## 8. BDD Scenario Summary

| No. | Feature  | Scenario                               | BDD validation                                         | Status |
|-----|----------|----------------------------------------|--------------------------------------------------------|--------|
| 1   | Combined | Open ixigo page                        | Then page title confirms ixigo loaded                  | Passed |
| 2   | Combined | Page elements are present              | Then From, To and Search controls are visible          | Passed |
| 3   | Combined | Search BOM to BLR flights              | Then matching results and visible flights remain       | Passed |
| 4   | Combined | Flight count and book buttons          | Then flights and at least two Book buttons are visible | Passed |
| 5   | Combined | Apply nonstop filter and sort by price | Then flights remain and parsed prices are sorted       | Passed |
| 6   | Combined | Print flight results                   | Then at least one result remains                       | Passed |
| 7   | Combined | Same source and destination            | Then bom-bom route is not accepted                     | Passed |
| 8   | Combined | Search without destination             | Then user remains off results page                     | Passed |
| 9   | Combined | Booking flow reaches traveller details | Then booking page is displayed after booking action    | Passed |
| 10  | E2E      | Open site                              | Then ixigo page should be loaded                       | Passed |
| 11  | E2E      | Login                                  | When mobile row is submitted and OTP wait completes    | Passed |
| 12  | E2E      | Search flights                         | Then matching flight results are displayed             | Passed |
| 13  | E2E      | Apply nonstop filter                   | Then at least one visible flight remains               | Passed |
| 14  | E2E      | Sort by price                          | Then at least one visible flight remains               | Passed |
| 15  | E2E      | Print results                          | Then at least one visible flight remains               | Passed |
| 16  | E2E      | Click book                             | When first visible flight is booked                    | Passed |
| 17  | E2E      | Booking page                           | Then booking page should be displayed                  | Passed |
| 18  | E2E      | Decline cancellation                   | When cancellation prompt is declined                   | Passed |
| 19  | E2E      | Fill traveller                         | Then traveller form fill assertion passes              | Passed |
| 20  | E2E      | Continue and confirm                   | When details are continued and confirmed               | Passed |
| 21  | E2E      | Add-ons page                           | Then add-ons page should be displayed                  | Passed |
| 22  | E2E      | Select seat                            | When a seat is selected if available                   | Passed |
| 23  | E2E      | Skip to payment                        | When add-ons are skipped to payment                    | Passed |
| 24  | E2E      | Payment page                           | Then payment page should be displayed                  | Passed |

## 9. Screenshot Evidence

The screenshots below are selected from the latest BDD run artifacts under logs. They are arranged in journey order so the report reads like the actual behaviour flow rather than a random screenshot dump.

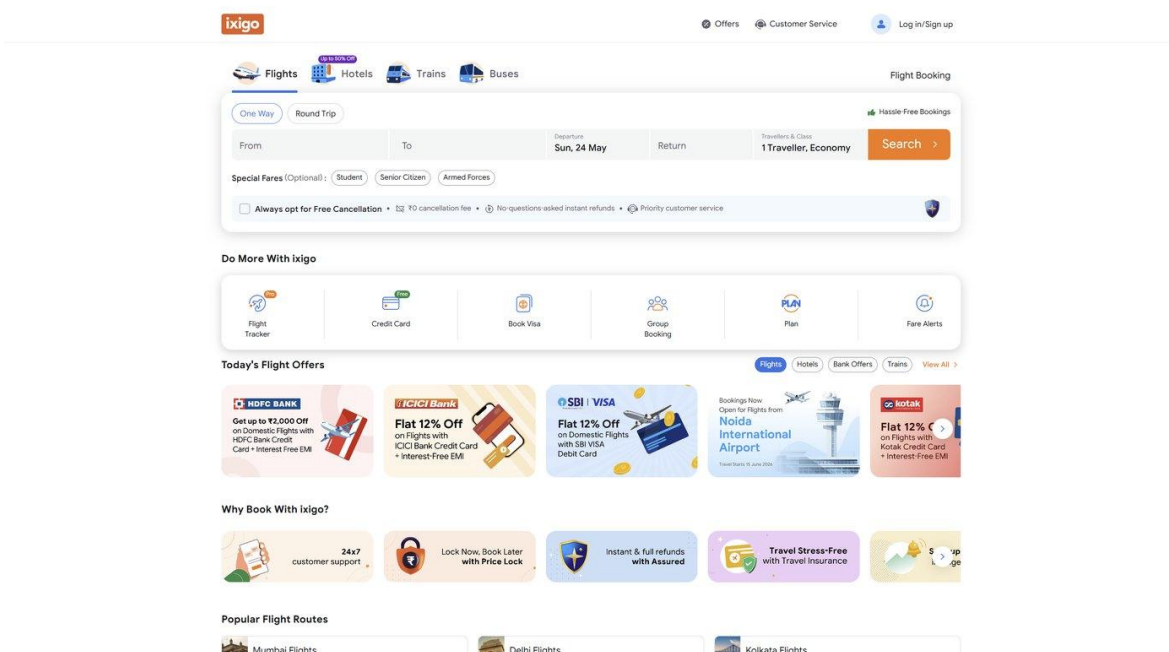


Figure 1. BDD scenario opens the ixigo flight page.

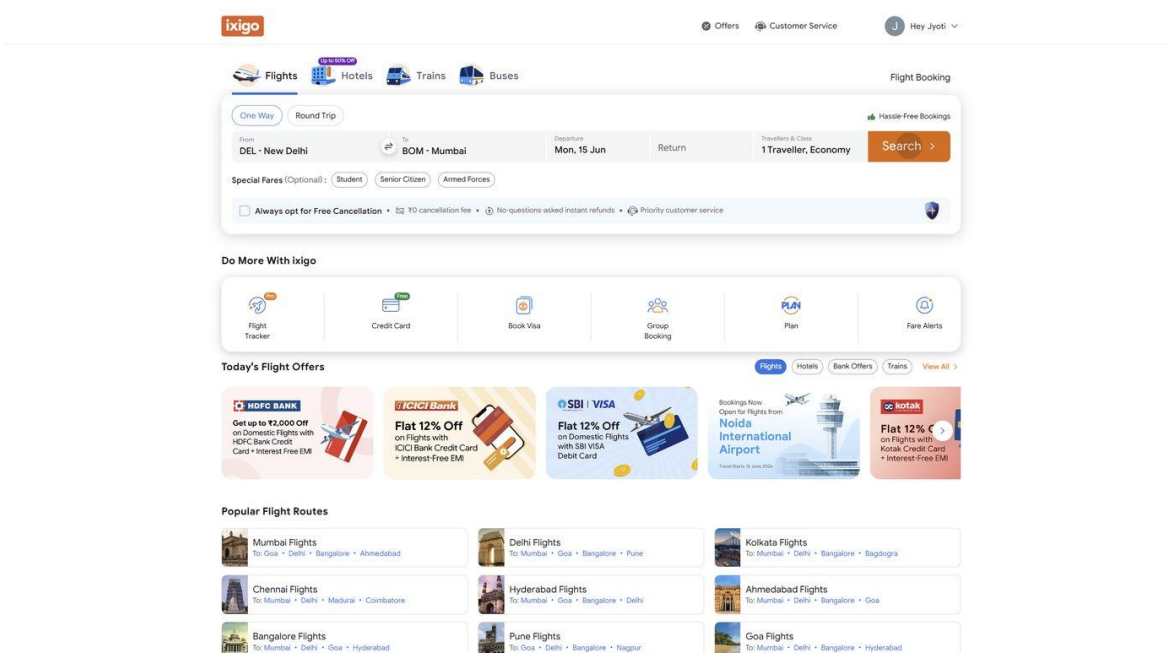


Figure 2. Route search executed from a BDD step.



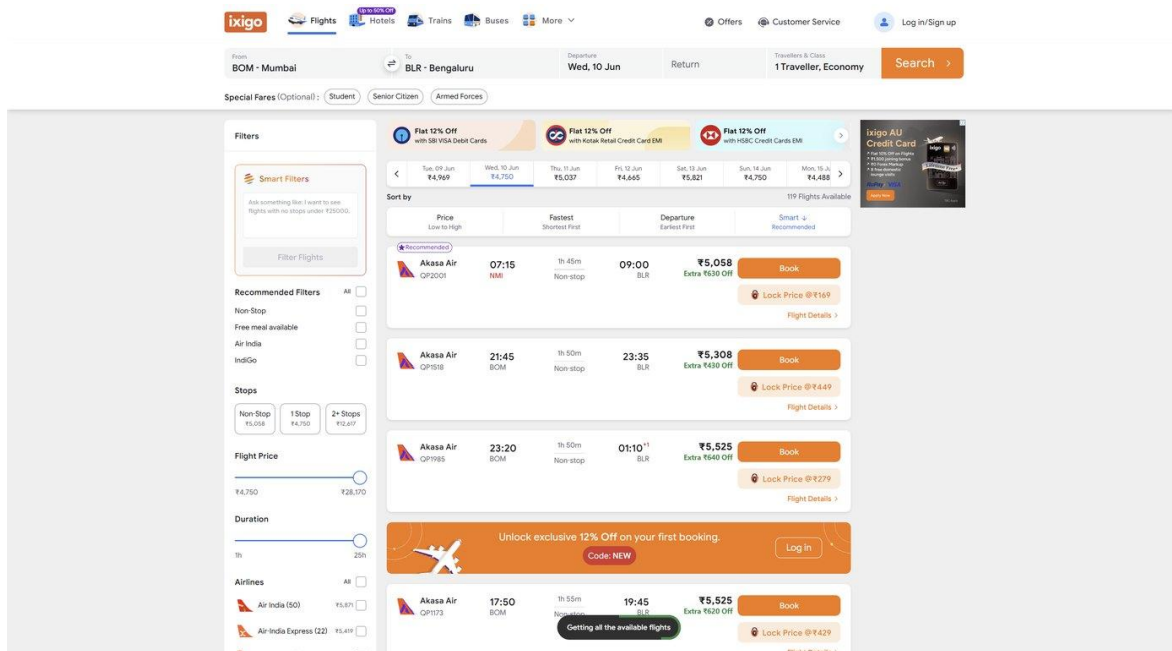


Figure 3. Combined flow validates that flight results are displayed.

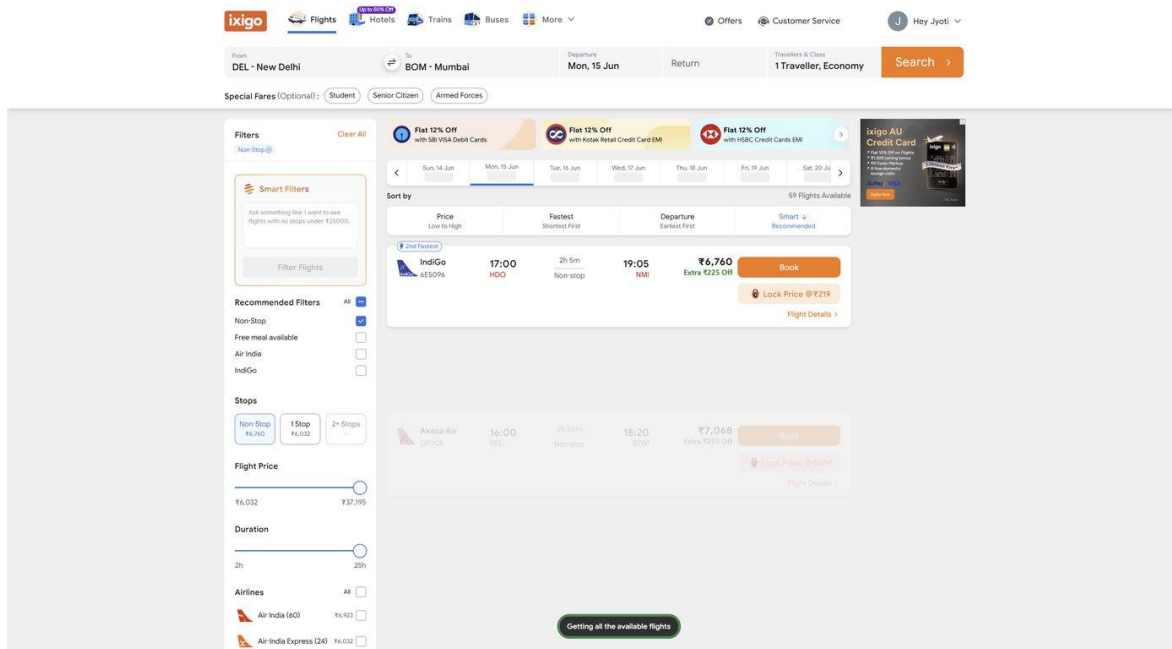


Figure 4. Non-stop filter applied during the behaviour flow.

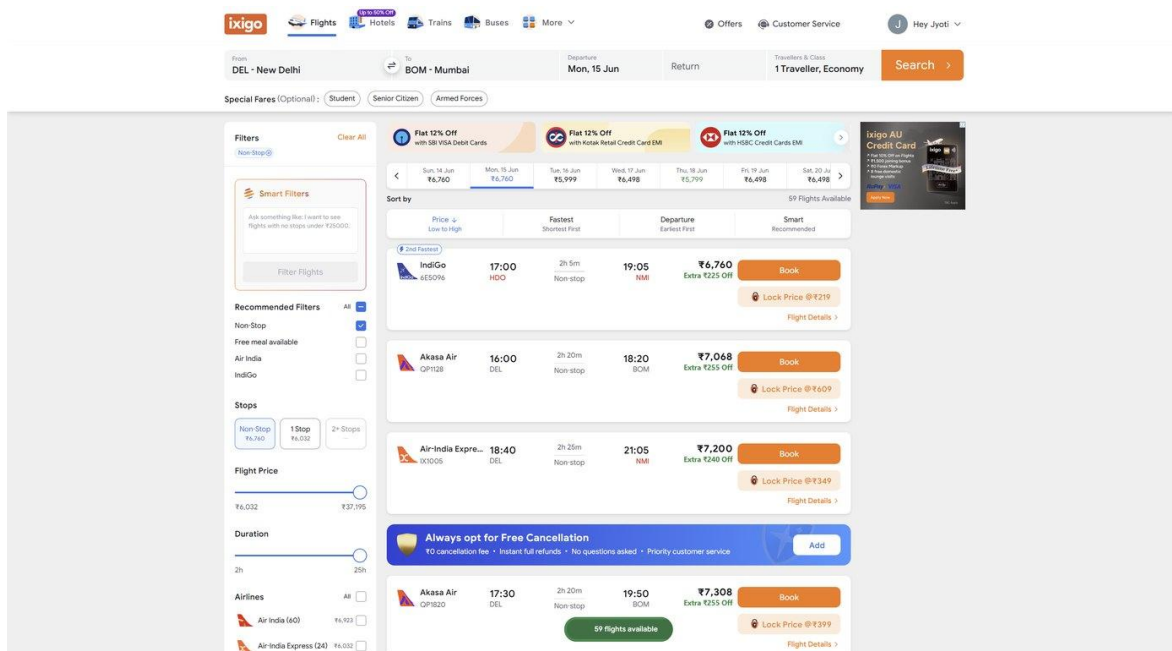


Figure 5. Price sorting evidence after filtering.

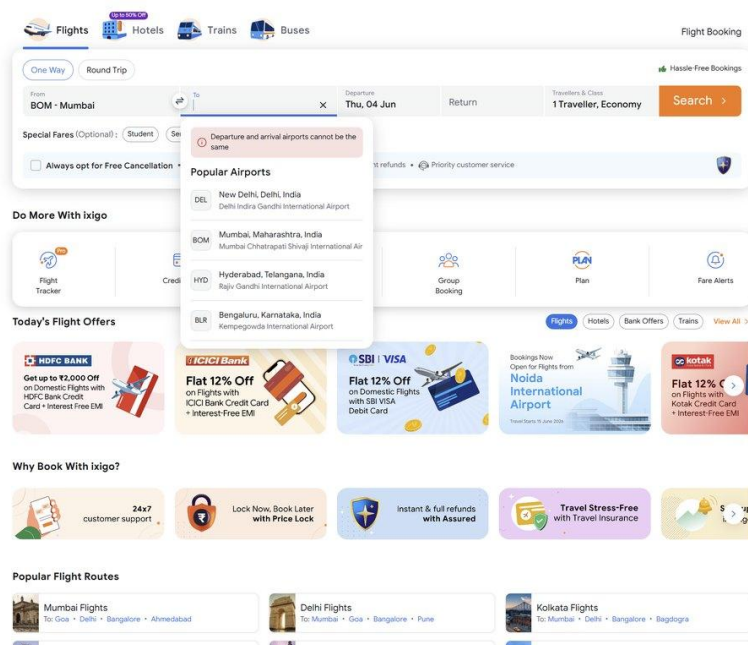


Figure 6. Negative behaviour confirms same source and destination are not accepted.

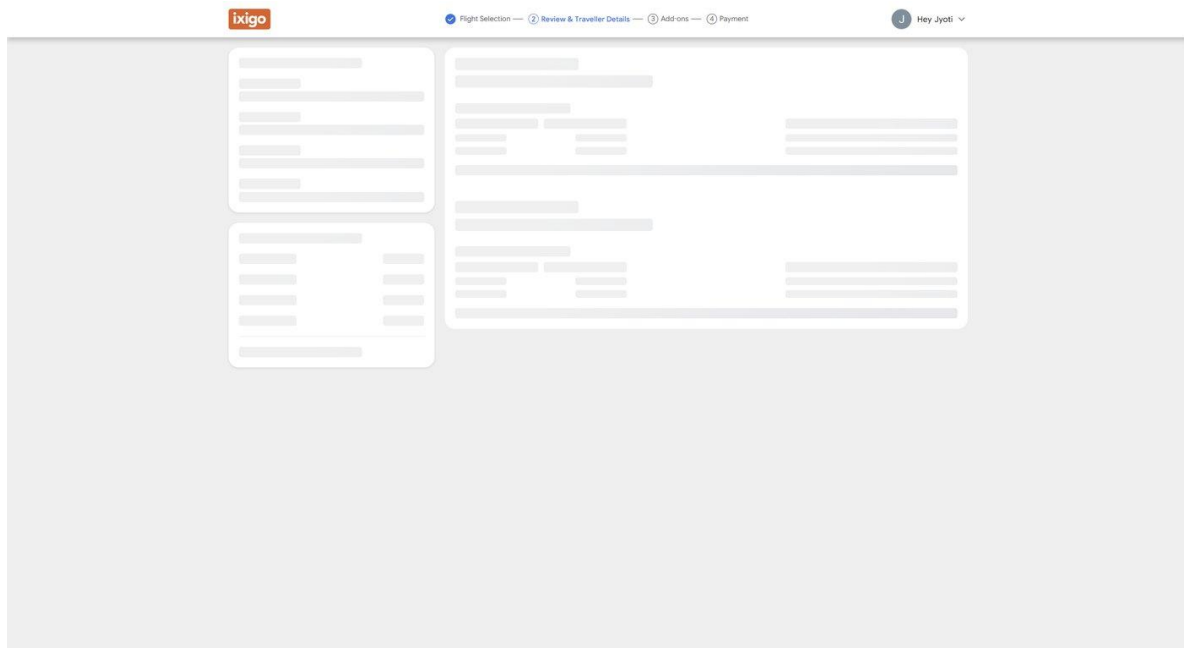


Figure 7. Booking action triggered from the result list.

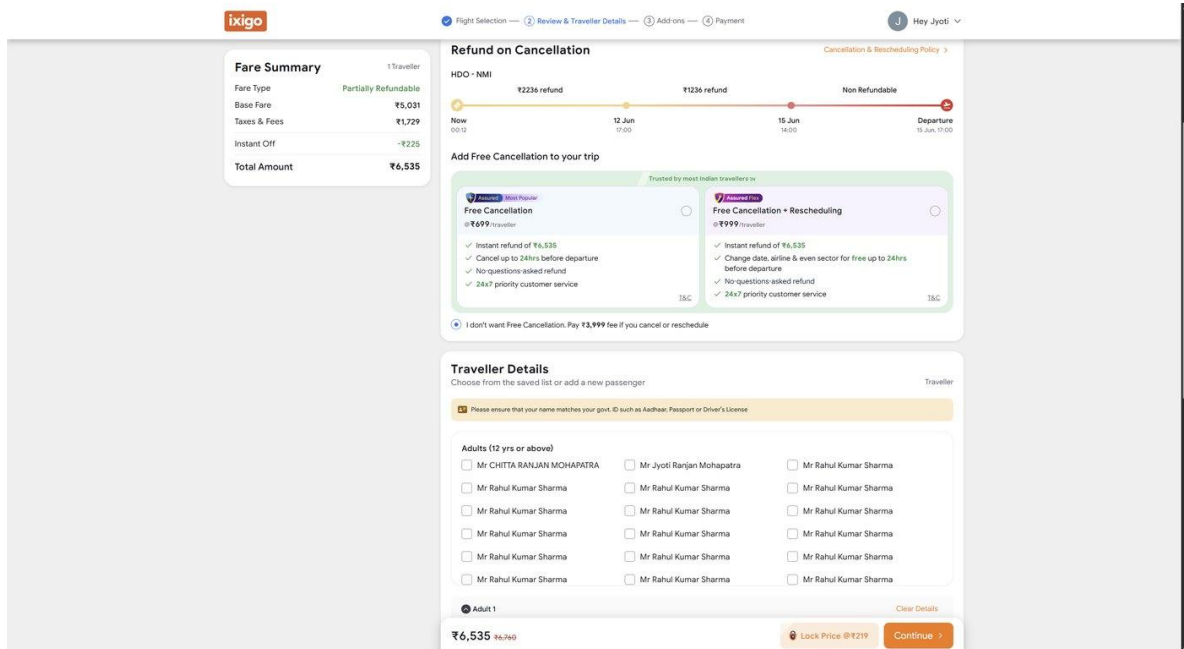


Figure 8. Cancellation prompt handled in the booking journey.

**ixigo** Flight Selection — 2 Review & Traveller Details — 3 Add-ons — 4 Payment Hey Jyoti

**Fare Summary** 1 Traveller  
Fare Type: Partially Refundable  
Base Fare: ₹5,031  
Taxes & Fees: ₹1,729  
Instant Off: -₹225  
Total Amount: ₹6,535

**Contact Details**  
Your ticket & flight information will be sent here  
Country Code: India (+91) Mobile Number: 9876543210 Email: rahul.sharma@example.com

**GST Details**  
To claim credit of GST charged by airlines, please enter your GST details  
☐ I would like to add my GST Number

**Billing Address**  
As per the latest govt. regulations, it's mandatory to provide your address.  
Pincode: 560001 Address: 42 MG Road Bengaluru  
City: Bengaluru State: Karnataka

By clicking on continue, I confirm that I have read, understood, and agree with the [Fare Rules](#), [Privacy Policy](#) and [Terms of Use](#).  
100% Safe Payment Process  
Verified by Visa SafeKey MasterCard SecureCode

₹6,535 ~~₹6,760~~ Lock Price @₹219 Continue

Figure 9. Traveller details filled from CSV data.

**ixigo** Flight Selection — 2 Review & Traveller Details — 3 Add-ons — 4 Payment Hey Jyoti

**Your Flight** One Way  
Mon, 15 Jun  
17:00 2h 5m 19:05  
Ghaziabad Non stop New Mumbai

**Travellers**  
1 Mr. Rahul Kumar Sharma

**Fare Summary** 1 Traveller  
Fare Type: Partially Refundable  
Base Fare: ₹5,031  
Taxes & Fees: ₹1,729  
Instant Off: -₹225  
Total Amount: ₹6,127

**Seat Map**  
Price: ₹425 - ₹475 ₹725 - ₹945  
F E D C B  
1 2 3 4 5 6 7  
Rahul Kumar S Select Seat

₹6,127 ~~₹6,760~~ Meal Selection

Figure 10. Add-ons page reached after confirming traveller details.

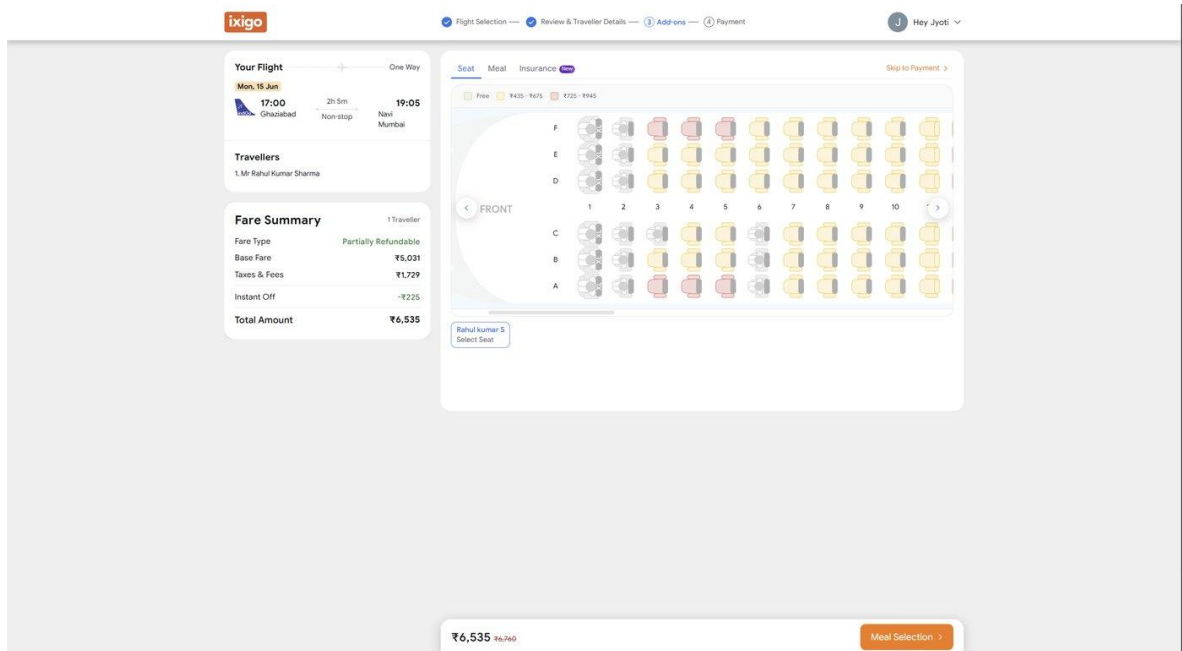


Figure 11. Seat selection behaviour executed where available.

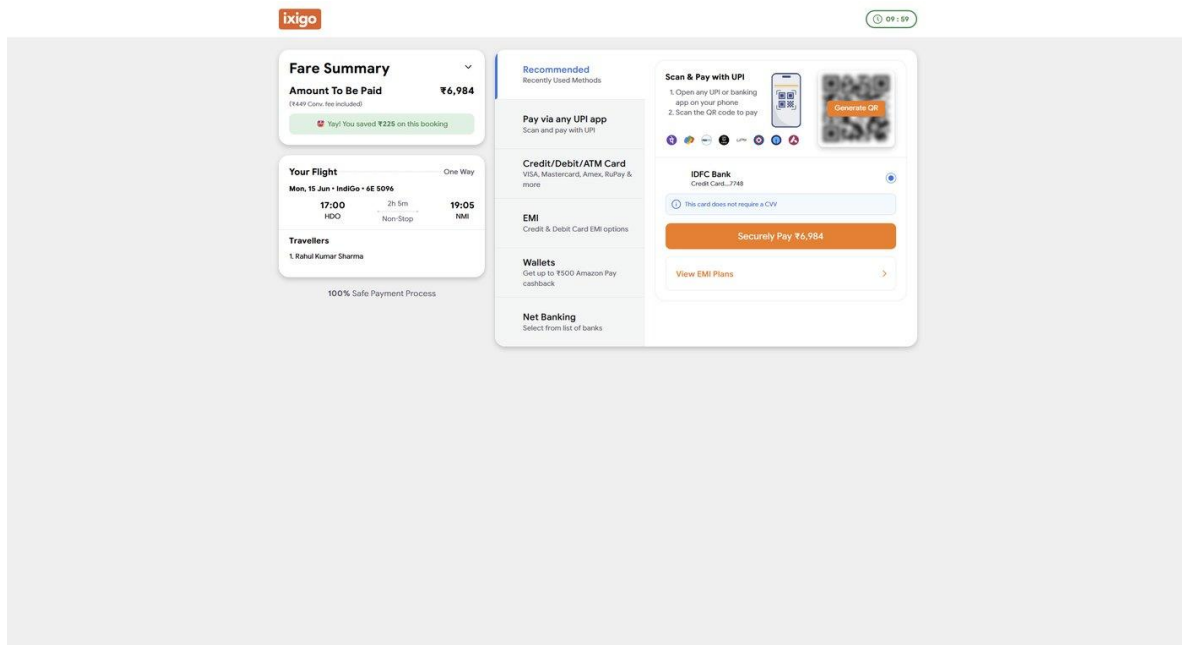


Figure 12. Payment page reached at the end of the BDD E2E flow.

## 10. Observations

- BDD is documented as the test design approach, while POM is documented as the automation code design pattern.
- The BDD layer makes the testing intent easier to explain because feature files read like user behaviour.

- The actual validations are still strong because Selenium assertions live inside the step-definition code.
- The two step folders give a clean evaluation structure: combined behaviours separately from the E2E booking journey.
- The browser lifecycle follows the Selenium style, with one feature-level browser and isolated browsers only where needed.
- The reporting hook keeps the project portable by opening HTML even when Allure CLI is missing.

## 11. Conclusion

The BDD project preserves the Selenium automation coverage while presenting it in a behaviour-driven format. The feature files communicate the business flow, the step folders keep implementation organised, and the assertions remain in Python where Selenium can verify the page state properly. The result is a Behave-based framework that is suitable for a capstone submission because it shows both testing discipline and readable behaviour coverage.